Digital Logic Design (DLD)
Complete Exam Preparation Guide

Course Code: CSE 211 | Based on Final Exam Pattern | All 14 Topics Covered

📚 Table of Contents

Topic 1: Number System (Conversions)

Key Concepts:

- **Binary (Base 2):** Digits 0, 1
- **Octal (Base 8):** Digits 0–7
- **Decimal (Base 10):** Digits 0–9
- **Hexadecimal (Base 16):** 0–9, A(10), B(11), C(12), D(13), E(14), F(15)
- **BCD:** Each decimal digit → 4-bit binary

Conversion Rules:

- **Integer part:** Divide by base, collect remainders bottom-up.
- **Fractional part:** Multiply by base, collect integer parts top-down.

Q1. Convert decimal value 3720 into (i) binary (ii) hex (iii) octal (iv) BCD

(i) Decimal to Binary $(3720)_{10}$

Division	Quotient	Remainder
3720 ÷ 2	1860	0 (LSB)
1860 ÷ 2	930	0
930 ÷ 2	465	0
465 ÷ 2	232	1
232 ÷ 2	116	0
116 ÷ 2	58	0
58 ÷ 2	29	0
29 ÷ 2	14	1
14 ÷ 2	7	0
7 ÷ 2	3	1
3 ÷ 2	1	1
1 ÷ 2	0	1 (MSB)

$\therefore (3720)_{10} = (111010001000)_{2}$

(ii) Decimal to Hexadecimal

Division	Quotient	Remainder
3720 ÷ 16	232	8

232 ÷ 16	14	8
14 ÷ 16	0	14 → E

$\therefore (3720)_{10} = (E88)_{16}$

(iii) Decimal to Octal

Division	Quotient	Remainder
3720 ÷ 8	465	0
465 ÷ 8	58	1
58 ÷ 8	7	2
7 ÷ 8	0	7

$\therefore (3720)_{10} = (7210)_{8}$

(iv) Decimal to BCD

Convert each digit individually to 4-bit binary:

3	7	2	0
0011	0111	0010	0000

$\therefore (3720)_{10} = (0011\ 0111\ 0010\ 0000)_{BCD}$

Q2. Convert decimal value 65.239 in (i) Binary (ii) Octal (iii) Hexadecimal (iv) BCD

(i) Decimal to Binary

Integer part (65):

Step	Quotient	Remainder
65 ÷ 2	32	1
32 ÷ 2	16	0
16 ÷ 2	8	0
8 ÷ 2	4	0
4 ÷ 2	2	0
2 ÷ 2	1	0
1 ÷ 2	0	1

$(65)_{10} = (1000001)_{2}$

Fractional part (0.239):

Step	Result	Integer
0.239 × 2	0.478	0
0.478 × 2	0.956	0
0.956 × 2	1.912	1
0.912 × 2	1.824	1
0.824 × 2	1.648	1
0.648 × 2	1.296	1

$\therefore (65.239)_{10} \approx (1000001.001111)_{2}$

(ii) Decimal to Octal

Integer (65): 65÷8=8 r1, 8÷8=1 r0, 1÷8=0 r1 → $(101)_8$

Fraction (0.239): 0.239×8=1.912(1); 0.912×8=7.296(7); 0.296×8=2.368(2); 0.368×8=2.944(2)

$\therefore (65.239)_{10} \approx (101.1722)_{8}$

(iii) Decimal to Hexadecimal

Integer (65): 65÷16=4 r1, 4÷16=0 r4 → $(41)_{16}$

Fraction (0.239): 0.239×16=3.824(3); 0.824×16=13.184(D); 0.184×16=2.944(2)

$\therefore (65.239)_{10} \approx (41.3D2)_{16}$

(iv) Decimal to BCD

6	5	.	2	3	9
0110	0101	.	0010	0011	1001

$\therefore (65.239)_{10} = (0110\ 0101\ .\ 0010\ 0011\ 1001)_{BCD}$

Q3. Convert $(1010.11)_2$ to Decimal, Octal and Hexadecimal

Binary to Decimal:

$= 1\times2^3 + 0\times2^2 + 1\times2^1 + 0\times2^0 + 1\times2^{-1} + 1\times2^{-2}$

$= 8 + 0 + 2 + 0 + 0.5 + 0.25 =$ **10.75**

$(1010.11)_2 = (10.75)_{10}$

Binary to Octal: Group in 3 from binary point:

`001 010 . 110` = $\mathbf{(12.6)_8}$ **Binary to Hex:** Group in 4 from binary point:

`1010 . 1100` = $\mathbf{(A.C)_{16}}$

Q4. Convert $(3FA)_{16}$ to decimal and binary

Hex to Decimal: $= 3\times16^2 + 15\times16^1 + 10\times16^0 = 768 + 240 + 10 = \mathbf{(1018)_{10}}$

Hex to Binary: Each hex digit → 4-bit:

3=0011, F=1111, A=1010 → $\mathbf{(001111111010)_2}$

Topic 2: Boolean Algebra & Minimization

Important Boolean Laws:

Law	OR form	AND form
Identity	A+0 = A	A·1 = A
Null	A+1 = 1	A·0 = 0
Idempotent	A+A = A	A·A = A
Complement	A+A' = 1	A·A' = 0
Commutative	A+B = B+A	AB = BA
Associative	A+(B+C)=(A+B)+C	A(BC)=(AB)C
Distributive	A(B+C)=AB+AC	A+BC=(A+B)(A+C)
Absorption	A+AB = A	A(A+B) = A
De Morgan's	(A+B)' = A'·B'	(AB)' = A'+B'
Consensus	A+A'B = A+B; A(A'+B) = AB	

Q1. Simplify F = AB̄ + (A+B+C·C̄) using Boolean Theorems

F = AB̄ + (A + B + C·C̄)
= AB̄ + (A + B + 0) [since C·C̄ = 0]
= AB̄ + A + B
= A(B̄ + 1) + B [factor A]
= A·1 + B [B̄+1 = 1]

F = A + B

Q2. Simplify F = XY(X + XȲ) + Y(X + Y)

F = XY(X + XȲ) + Y(X + Y)
= XY·X + XY·XȲ + YX + Y·Y
= XY + X·X·Y·Ȳ + XY + Y [Y·Ȳ = 0]
= XY + 0 + XY + Y
= XY + Y
= Y(X + 1)

F = Y

Q3. Simplify F = A'B'C + A'BC + AB'C' + AB'C + ABC

F = A'B'C + A'BC + AB'C' + AB'C + ABC
Group: (A'B'C + A'BC) + (AB'C' + AB'C) + ABC
= A'C(B'+B) + AB'(C'+C) + ABC
= A'C + AB' + ABC
= A'C + A(B' + BC)
= A'C + A(B' + C) [absorption: B'+BC = B'+C]
= A'C + AB' + AC
= C(A'+A) + AB'
= C + AB'

```
F = C + AB'
```

Q4. Simplify F = (A+B)(A+B')(A'+C)

(A+B)(A+B') = A + BB' = A + 0 = A
F = A·(A'+C) = AA' + AC = 0 + AC = AC

```
F = AC
```

Topic 3: Truth Table

A **truth table** lists the output of a logic function for every possible combination of inputs. For n inputs there are 2^n rows.

Q1. Construct truth table for F = AB + A'C

A	B	C	AB	A'C	F = AB+A'C
0	0	0	0	0	0
0	0	1	0	1	1
0	1	0	0	0	0
0	1	1	0	1	1
1	0	0	0	0	0
1	0	1	0	0	0
1	1	0	1	0	1
1	1	1	1	0	1

Q2. Construct truth table for F = (A⊕B) + C

A	B	C	A⊕B	F
0	0	0	0	0
0	0	1	0	1
0	1	0	1	1
0	1	1	1	1
1	0	0	1	1
1	0	1	1	1
1	1	0	0	0
1	1	1	0	1

Q3. From Truth Table → Simplify T_1, T_2 (from your exam)

A	B	C	T_1	T_2

0	0	0	1	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	0	1
1	0	1	0	1
1	1	0	0	1
1	1	1	0	1

T_1 = 1 for minterms 0,1,2 → T_1 = A'B'C' + A'B'C + A'BC'
= A'B'(C'+C) + A'BC' = A'B' + A'BC' = A'(B' + BC') = A'(B' + C')

T_1 = A'(B' + C') = A' B' + A' C' (3 literals)

T_2 = 1 for minterms 3,4,5,6,7
= A + A'BC
= A + BC (consensus theorem)

T_2 = A + BC (3 literals)

Topic 4: SOP & POS Form (Interchange)

SOP (Sum of Products): OR of AND terms. Example: F = AB + A'C + BC
POS (Product of Sums): AND of OR terms. Example: F = (A+B)(A'+C)(B+C)
Canonical SOP: Each product has all variables (minterms).
Canonical POS: Each sum has all variables (maxterms).
Interchange Rule: SOP minterms not present become POS maxterms (and vice versa).

Q1. Convert F(A,B,C) = Σm(1, 3, 5, 7) to POS form

Total minterms for 3 variables = $2^3 = 8$ (m0..m7).
Missing minterms = {0, 2, 4, 6} → these are the maxterms in POS.

```
F(A,B,C) = ΠM(0, 2, 4, 6)
```

Expanded:
M_0 = (A+B+C), M_2 = (A+B'+C), M_4 = (A'+B+C), M_6 = (A'+B'+C)
F = (A+B+C)(A+B'+C)(A'+B+C)(A'+B'+C)

Q2. Convert F = AB + A'C to canonical SOP

F = AB(C+C') + A'C(B+B')
= ABC + ABC' + A'BC + A'B'C
Minterms: ABC=7, ABC'=6, A'BC=3, A'B'C=1

```
F(A,B,C) = Σm(1, 3, 6, 7)
```

Q3. Convert F = (A+B)(B+C) to canonical POS

(A+B) = (A+B+CC') = (A+B+C)(A+B+C')
(B+C) = (B+C+AA') = (A+B+C)(A'+B+C)
F = (A+B+C)(A+B+C')(A'+B+C)
Maxterms: (A+B+C)=M_0, (A+B+C')=M_1, (A'+B+C)=M_4

```
F(A,B,C) = ΠM(0,1,4)
```

Q4. Convert F(A,B,C,D) = ΠM(0,2,5,8,10) to SOP form

For 4 variables: total minterms = 16.
F = Σm of all not in {0,2,5,8,10}

F(A,B,C,D) = Σm(1,3,4,6,7,9,11,12,13,14,15)

Topic 5: Minterm, Maxterm, Literals (Definitions)

Literal:

A **literal** is a single variable or its complement (e.g., A, A', B, B'). The expression **F = A'BC + AB'** has 5 literals.

Minterm (Standard Product):

A product term containing **all variables** of the function in either complemented or uncomplemented form. For n variables there are 2^n minterms ($m_0..m_{2^n-1}$).

Maxterm (Standard Sum):

A sum term containing **all variables** of the function in either complemented or uncomplemented form. For n variables there are 2^n maxterms.

Relationship:

$m_i' = M_i$ (complement of minterm i = maxterm i)

Minterms & Maxterms Table for 3 variables

A	B	C	Minterm	Designation	Maxterm	Designation
0	0	0	A'B'C'	m_0	A+B+C	M_0
0	0	1	A'B'C	m_1	A+B+C'	M_1
0	1	0	A'BC'	m_2	A+B'+C	M_2
0	1	1	A'BC	m_3	A+B'+C'	M_3
1	0	0	AB'C'	m_4	A'+B+C	M_4
1	0	1	AB'C	m_5	A'+B+C'	M_5
1	1	0	ABC'	m_6	A'+B'+C	M_6
1	1	1	ABC	m_7	A'+B'+C'	M_7

Q1. Identify number of literals in F = A'B + ABC' + B'CD

A'B → 2 literals (A', B)
ABC' → 3 literals
B'CD → 3 literals
Total = 2 + 3 + 3 = 8 literals

Q2. Express F = AB + A'C as sum of minterms (3 variables)

F = AB(C+C') + A'C(B+B') = ABC + ABC' + A'BC + A'B'C

$F = m_1 + m_3 + m_6 + m_7 = \Sigma(1,3,6,7)$

Topic 6: K-Map (Karnaugh Map)

Rules of K-Map grouping:

1. Groups must contain 1, 2, 4, 8, 16 cells (powers of 2).
2. Groups must be horizontal/vertical (not diagonal).
3. Each group must be as **large** as possible.
4. Each "1" must be covered by at least one group.
5. Wrap-around (sides and corners) is allowed.
6. Use **don't cares** (×) only if they help form bigger groups.

Q1. Simplify F(A,B,C,D) = Σm(4,5,7,8,10,11,13,14) + Σd(0,1,2)

4-variable K-map (rows AB; columns CD with Gray code):

AB \ CD	00	01	11	10
00	×	×	0	×
01	1	1	1	0
11	0	1	0	1
10	1	0	1	1

Groupings:

- Group of 4: m4, m5, m7 + don't cares m0, m1 → row AB=01, plus column CD=01... → simplifies to **B** region partially. Better: {m0,m1,m4,m5} = A'C' (using don't cares 0,1)
- Group of 2: m5, m7 = A'BD (covered above better via grouping with d0,d1: we get A'C')
- Group of 4 (corners): m8,m10 + don't cares not present here. Take {m8,m10,m14,m... }

Optimal groups:

1. {m4, m5, d0, d1} = A'C' (cells where A=0,C=0)
2. {m5, m7, m13, ...} → take {m5,m7} with {m13, m15-no}; use {m4,m5,m7,m... } = A'BD' (m4,m5) + A'BD (m5,m7) — combine: {m4,m5,m7,d... } → actually {m5,m7,m13,m15-no} → use BD: {m5, m7, m13, m15} but m15=0, so just {m5,m7}=A'BD? No m13 also has D combinations.

Simplified Method (after grouping carefully):

```
F = A'C' + BD' + B'C + AD' (one valid minimal SOP)
```

Note: Multiple equivalent minimum SOP forms exist depending on grouping choice.

Q2. Simplify F(A,B,C,D) = Σm(5,6,7,12,14) + d(3,9,11,15)

AB \ CD	00	01	11	10
00	0	0	×	0
01	0	1	1	1
11	1	0	×	1
10	0	×	×	0

Groupings:

- Group of 4: {m5, m7, d9, d11, d15, m13(no)} → take {m5, m7, m13(no)... } : {m5, m7, d11, d15} → wait m13=0. Take column CD=11 (rows 00,01,11,10) = {d3, m7, d15, d11} → all 1/× → simplifies to **CD**
- Group of 4: {m6, m7, m14, m... } → take {m6, m7, m14, m15(d)} = BC
- Group of 2: {m12, m14} = AB·D' → can extend to {m12,m14,d... } → {m12,m14} only = ABD'

```
F = CD + BC + ABD'
```

Q3. Simplify F(A,B,C,D) = Σm(4,5,7,8,10,11,13,14) + d(0,1,2) [from your exam]

This is repeated for clarity with full Logic Diagram. **K-Map filled:**

AB \ CD	00	01	11	10
00	×(0)	×(1)	0(3)	×(2)
01	1(4)	1(5)	1(7)	0(6)
11	0(12)	1(13)	0(15)	1(14)
10	1(8)	0(9)	1(11)	1(10)

Optimal grouping:

1. Quad {0,1,4,5} = A'C'
2. Quad {5,7,13, 15(=0 NO)} → use {5,7} + {13} pair: pair {5,13}=BC'D , extend to quad: {1,5,9(0 no),13} =NO. So take pair {5,7}=A'BD and pair {13}=ABC'D? Better: {1,5} d=A'C'D extends; we have A'C' already.
3. Quad {8,10,11,? } → {8,10} pair AB'D'; {10,11}=AB'C; extend {8,10,? ,? } with wraparound: {0,2,8,10}=B'D' (using d0,d2)
4. Pair {10,11}=AB'C
5. Pair {13,? } → covered

Final minimal SOP:

```
F = A'C' + B'D' + AB'C + BC'D
```

Logic Diagram: Use 4 AND gates (2-input or 3-input) feeding a 4-input OR gate. Inputs A, B, C, D, plus their complements via NOT gates.

Topic 7: Drawing Logic Diagram of Expression

Steps to draw a logic diagram:

1. Identify variables and their complements (use NOT gates).
2. For each AND term → use AND gate(s).
3. Combine all terms with an OR gate.
4. Output of OR gate = function output F.

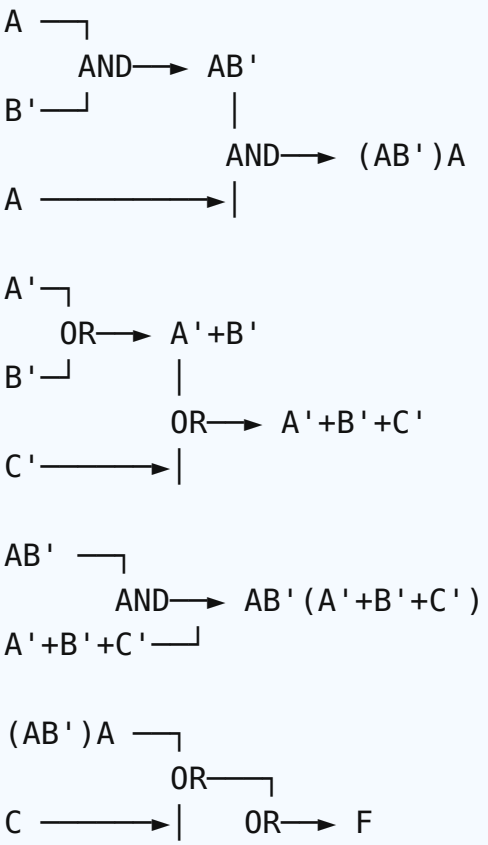
Q1. Without simplification using only 2-input basic gates, sketch the Logic Diagram for:

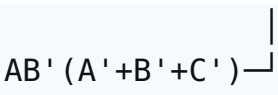
$F = (A\,\bar{B})A + C + (A\,\bar{B})(\bar{A} + \bar{B} + \bar{C})$

Step-wise breakdown:

1. Compute $A\,\bar{B}$ → AND gate (A, B').
2. Compute $(A\,\bar{B})\cdot A$ → AND gate (output of step1, A).
3. Compute $\bar{A} + \bar{B}$ → OR gate. Then OR with $\bar{C}$ → another OR gate.
4. Compute $(A\,\bar{B})\cdot(\bar{A} + \bar{B} + \bar{C})$ → AND gate.
5. Final OR: [Step2] + C + [Step4] → multiple 2-input ORs.

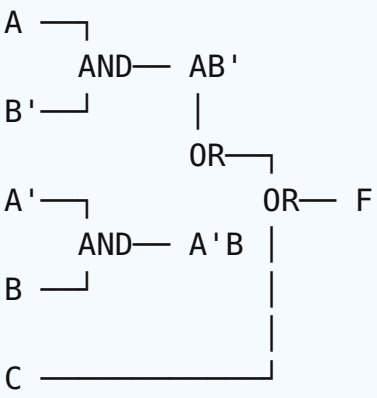
Logic Diagram (Text Form):





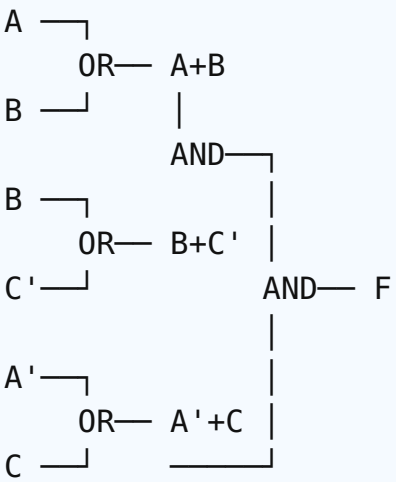
Gate Count: 3 NOT (for B', A', C'), 3 AND gates, 4 OR gates.

Q2. Draw Logic Diagram for F = AB' + A'B + C using 2-input gates



Need 2 NOT gates (A', B'), 2 AND gates, 2 OR gates.

Q3. Draw the logic diagram of F = (A+B)(B+C')(A'+C)



Uses 3 OR gates + 2 AND gates + 2 NOT gates.

Topic 8: Combinational Logic Circuits

8.1 Half Adder

Adds two 1-bit numbers A and B. Outputs **Sum** and **Carry**.

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$$\text{Sum} = A \oplus B$$
$$\text{Carry} = A \cdot B$$

8.2 Full Adder

Adds three 1-bit inputs (A, B, C_{in}). Outputs Sum and C_{out}.

A	B	C_{in}	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

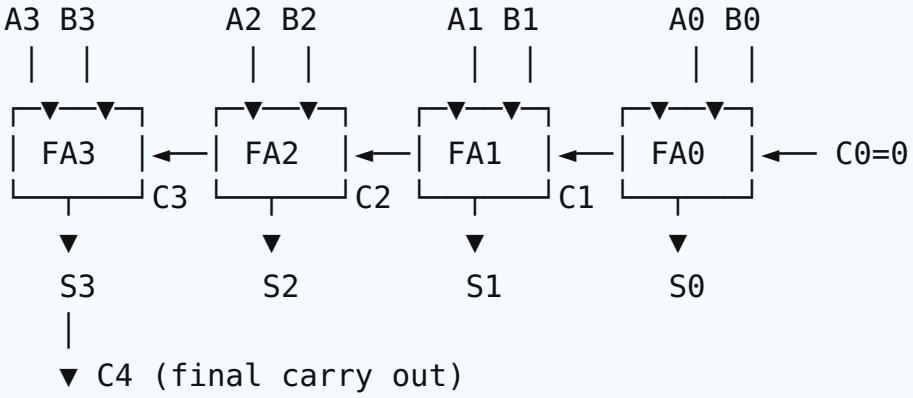
$$\text{Sum} = A \oplus B \oplus C_{in}$$
$$C_{out} = AB + (A \oplus B)C_{in} = AB + BC_{in} + AC_{in}$$

Q. Explain how 4-bit (Binary) Parallel Adder works. Add 1011 and 0101.

A 4-bit parallel adder consists of **4 full adders** connected in cascade. The carry out (C_{out}) of one stage becomes carry in (C_{in}) of the next. Initial carry $C_0 = 0$. **Adding A = 1011 and B = 0101:**

Stage	A_i	B_i	C_{in}	Sum	C_{out}
FA0 (LSB)	1	1	0	0	1
FA1	1	0	1	0	1
FA2	0	1	1	0	1
FA3 (MSB)	1	0	1	0	1

Result = $C_4\ S_3S_2S_1S_0$ = **1 0000** = 16_{10} ✓ (since 11+5=16). **Block Diagram:**



8.3 Half Subtractor

Subtracts B from A. Outputs **Difference** and **Borrow**.

A	B	D (Diff)	B_{out}
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

```
Difference = A ⊕ B
Borrow = A' · B
```

8.4 Full Subtractor

Subtracts B and previous borrow B_{in} from A.

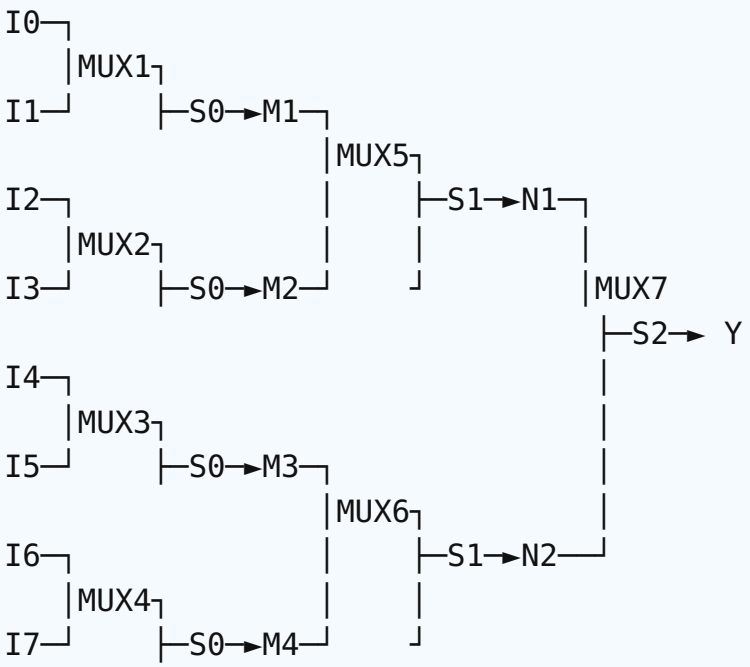
```
Difference = A ⊕ B ⊕ Bin
Bout = A'B + (A'+B)·Bin = A'B + A'Bin + BBin
```

8.5 Multiplexer (MUX)

A MUX selects one of 2^n inputs based on n select lines and routes it to a single output.
2×1 MUX: $Y = S' \cdot I_0 + S \cdot I_1$
4×1 MUX: $Y = S_1'S_0'I_0 + S_1'S_0I_1 + S_1S_0'I_2 + S_1S_0I_3$
8×1 MUX: Uses 3 select lines.

Q. Design 8×1 MUX using 2×1 MUX. Draw circuit diagram and Truth Table.

An 8×1 MUX has 8 data inputs (I_0–I_7) and 3 select lines (S_2, S_1, S_0). It can be built using **seven 2×1 MUXes** arranged in a tree: **Stage 1:** 4 MUXes (selected by S_0): pairs (I_0,I_1), (I_2,I_3), (I_4,I_5), (I_6,I_7) → outputs M1, M2, M3, M4.
Stage 2: 2 MUXes (selected by S_1): pairs (M1,M2), (M3,M4) → outputs N1, N2.
Stage 3: 1 MUX (selected by S_2): inputs N1, N2 → final output Y.



Truth Table:

S_2	S_1	S_0	Y
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	

			I_6
1	1	1	I_7

Q. Implement F(A,B,C,D) = Σ(1,2,4,5,6,9,11,13,15) using a Multiplexer

Use an **8×1 MUX**. Connect A, B, C as select lines (S_2, S_1, S_0). The 8 data inputs depend on D.

A	B	C	Minterms (with D=0,1)	F(D=0)	F(D=1)	Input I
0	0	0	m0(0), m1(1)	0	1	D
0	0	1	m2(1), m3(0)	1	0	D'
0	1	0	m4(1), m5(1)	1	1	1
0	1	1	m6(1), m7(0)	1	0	D'
1	0	0	m8(0), m9(1)	0	1	D
1	0	1	m10(0), m11(1)	0	1	D
1	1	0	m12(0), m13(1)	0	1	D
1	1	1	m14(0), m15(1)	0	1	D

Connections:

I_0=D, I_1=D', I_2=1, I_3=D', I_4=D, I_5=D, I_6=D, I_7=D

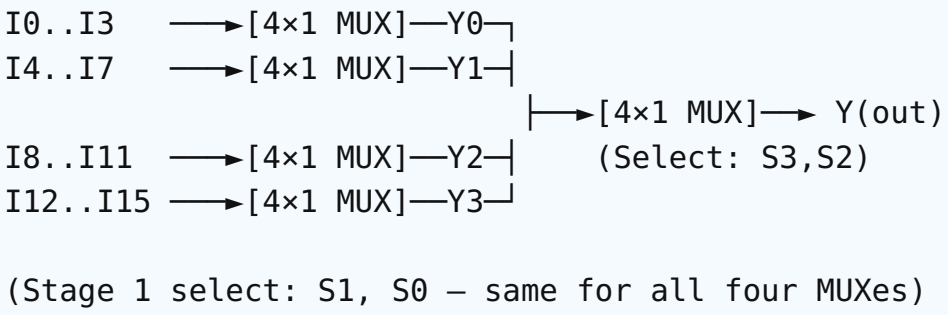
S_2=A, S_1=B, S_0=C

Q. Construct a 16×1 MUX using 4×1 MUXes

We need **five 4×1 MUXes** arranged in 2 levels.

Stage 1: Four 4×1 MUXes (each handles 4 inputs), all share select lines S_1, S_0.

Stage 2: One 4×1 MUX combines the 4 outputs, with select lines S_3, S_2.



Total select lines = $S_3S_2S_1S_0$ (4 bits) → 16 combinations.

8.6 Encoder & Decoder

Encoder: Has 2^n inputs and n outputs. Converts active input → binary code.
Decoder: Has n inputs and 2^n outputs. Converts binary code → 1 active output line.
A 2-to-4 decoder outputs are: D_0=A'B', D_1=A'B, D_2=AB', D_3=AB.

Topic 9: Decimal Codes (BCD, Excess-3, Error Detection)

BCD (Binary Coded Decimal): Each decimal digit (0–9) → 4-bit binary. Codes 1010–1111 are invalid.
Excess-3 Code: Add 3 to decimal digit, then convert to 4-bit binary. (e.g., 5 → 5+3=8 → 1000)
Error Detection Code: Parity bit added to detect errors. Even parity → total 1's = even; Odd parity → total 1's = odd.

BCD vs Excess-3 Table

Decimal	BCD	Excess-3
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100

Q1. Convert 947 to BCD and Excess-3 code

BCD: 9=1001, 4=0100, 7=0111 → **1001 0100 0111**
Excess-3: 9+3=12=1100, 4+3=7=0111, 7+3=10=1010 → **1100 0111 1010**

Q2. Add even parity bit to 1011010

Number of 1's = 4 (already even). So parity bit = 0.
Encoded: 0 1011010 (parity bit at MSB)

Q3. Add odd parity bit to 1100110

Number of 1's = 4 (even). For odd parity, parity = 1.
Encoded: 1 1100110

Topic 10: Complements – r's and (r-1)'s

For a number system with base r:

- **(r−1)'s complement:** Subtract each digit from (r−1).
- **r's complement:** (r−1)'s complement + 1.

Common cases:

System	(r−1)'s	r's
Binary (r=2)	1's complement	2's complement
Decimal (r=10)	9's complement	10's complement
Octal (r=8)	7's complement	8's complement
Hex (r=16)	15's complement	16's complement

Q1. Find 1's and 2's complement of 1011001

1's complement: Flip all bits → **0100110**
2's complement: 1's + 1 → 0100110 + 1 = **0100111**

Q2. Find 9's and 10's complement of 3250

9's complement: Subtract each digit from 9.
9999 − 3250 = **6749**
10's complement: 9's + 1 = 6749 + 1 = **6750**

Q3. Find 7's and 8's complement of $(567)_8$

7's complement: 777 − 567 = **210**
8's complement: 210 + 1 = **211**

Q4. Find 15's and 16's complement of $(3A8)_{16}$

15's: FFF − 3A8 = **C57**
16's: C57 + 1 = **C58**

Topic 11: Subtraction Using Complement

Procedure (using r's complement):

1. Find r's complement of subtrahend.
2. Add it to minuend.
3. If end carry → result is positive (discard carry).
4. If no end carry → result is negative; take r's complement of result and put a minus sign.

Q1. Subtract $(447)_{16}$ from $(317)_{10}$ using 2's complement. Verify in decimal.

Step 1: Convert to common base (binary).
$(447)_{16}$ = 4=0100, 4=0100, 7=0111 → 0100 0100 0111 = $(1095)_{10}$
$(317)_{10}$ = ? Convert to binary:
317÷2: 158 r1, 79 r0, 39 r1, 19 r1, 9 r1, 4 r1, 2 r0, 1 r0, 0 r1 → $(100111101)_2$
So we are computing: 317 − 1095 = −778 (decimal verification) **Step 2: Make both numbers same bit-width (12 bits).**
Minuend: 317 = 0001 0011 1101
Subtrahend: 1095 = 0100 0100 0111 **Step 3: 2's complement of subtrahend (1095).**
1's: 1011 1011 1000
2's: 1011 1011 1001 **Step 4: Add.**

```
  0001 0011 1101
+ 1011 1011 1001
-----------------
  1100 1111 0110   (no end carry)
```

Step 5: No carry → result is negative. Take 2's complement of result.
1's of 1100 1111 0110 = 0011 0000 1001
+1 = 0011 0000 1010 = $(778)_{10}$

Result = $-(778)_{10}$

Verification: 317 − 1095 = −778 ✓

Q2. Subtract 1010 − 1101 using 2's complement

2's comp of 1101 = 0010 + 1 = 0011
1010 + 0011 = 1101 (no end carry, MSB=1 → negative)
2's comp of 1101 = 0010 + 1 = 0011 = 3
Result = −3 (verify: 10−13=−3 ✓)

Q3. Subtract 7264 − 4321 using 10's complement

10's comp of 4321 = (9999−4321)+1 = 5678+1 = 5679
7264 + 5679 = 12943
End carry present → positive. Discard carry → **2943**
Verify: 7264 − 4321 = 2943 ✓

Q4. Subtract 4321 − 7264 using 10's complement

10's comp of 7264 = (9999−7264)+1 = 2735+1 = 2736
4321 + 2736 = 7057
No end carry → negative. 10's comp of 7057 = (9999−7057)+1 = 2942+1 = 2943
Result = −2943

Topic 12: Full Adder Building Using Decoder

A 3-to-8 decoder has 3 inputs (X, Y, Z) and 8 outputs (D_0–D_7). Each output corresponds to one minterm. Any function can be built by ORing the appropriate output lines.

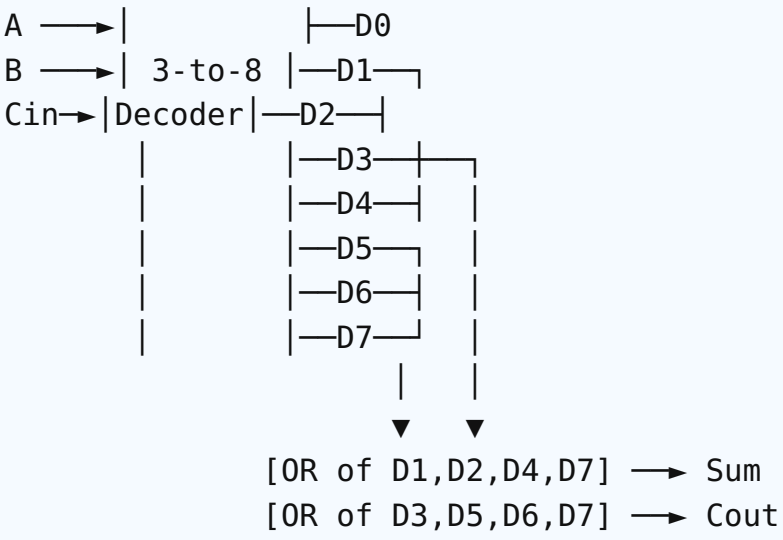
Q. Implement Full Adder using a 3-to-8 Decoder.

Truth Table:

Minterm	A	B	C_{in}	Sum	C_{out}
0	0	0	0	0	0
1	0	0	1	1	0
2	0	1	0	1	0
3	0	1	1	0	1
4	1	0	0	1	0
5	1	0	1	0	1
6	1	1	0	0	1
7	1	1	1	1	1

Sum = Σm(1, 2, 4, 7)

C_{out} = Σm(3, 5, 6, 7) **Implementation:**



Topic 13: 2-bit Magnitude Comparator

A magnitude comparator compares two binary numbers and produces three outputs: A>B, A=B, A<B. For 2-bit comparator: A = A_1A_0, B = B_1B_0.

Q. Design a 2-bit Magnitude Comparator. Show Truth Table, K-maps, and Logic Diagram.

Truth Table:

A_1	A_0	B_1	B_0	A>B	A=B	A<B
0	0	0	0	0	1	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	1	0	0
0	1	0	1	0	1	0
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	1	0
1	0	1	1	0	0	1
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	1	0

Simplified Boolean Expressions (using K-map):

$$A>B = A_1B_1' + A_0B_1'B_0' + A_1A_0B_0'$$

$$A=B = (A_1 \odot B_1)\cdot(A_0 \odot B_0) = (A_1 \oplus B_1)'\cdot(A_0 \oplus B_0)'$$

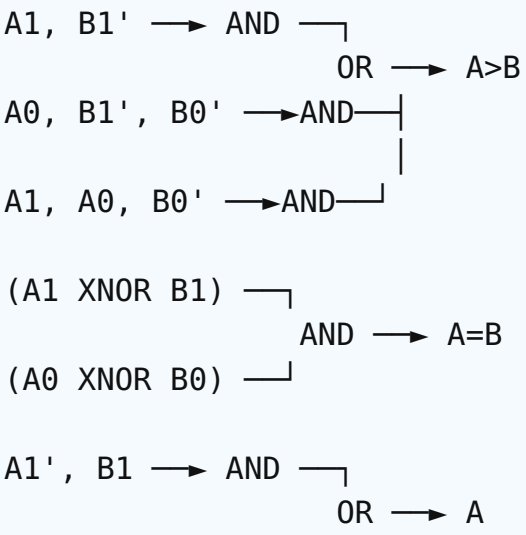
$$A<B = A_1'B_1 + A_0'B_1B_0 + A_1'A_0'B_0$$

K-Map for A>B (taking A_1A_0 rows, B_1B_0 columns):

$A_1A_0 \backslash B_1B_0$	00	01	11	10

00	0	0	0	0
01	1	0	0	0
11	1	1	0	1
10	1	1	0	0

Group: A_1B_1' (covers four cells in row 10,11 col 00,01) + $A_1A_0B_0'$ + $A_0B_1'B_0'$ **Logic Diagram:**



Topic 14: Priority Encoder

A priority encoder assigns priority to inputs. When more than one input is active, only the highest-priority input is encoded. Often includes a Valid (V) output.

Q. Design a 4-input Priority Encoder. Truth table, K-map, and circuit.

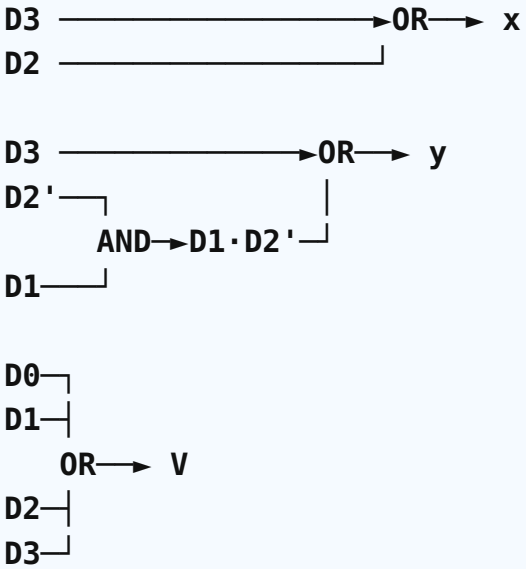
Inputs: D_3, D_2, D_1, D_0 (D_3 = highest priority)
Outputs: x, y (binary code), V (valid) Truth Table:

D_3	D_2	D_1	D_0	x	y	V
0	0	0	0	×	×	0
0	0	0	1	0	0	1
0	0	1	×	0	1	1
0	1	×	×	1	0	1
1	×	×	×	1	1	1

Boolean Expressions (after K-map):

$$x = D_2 + D_3$$
$$y = D_3 + D_1D_2'$$
$$V = D_0 + D_1 + D_2 + D_3$$

Logic Diagram:



Topic 15: Universality of NAND and NOR Gates

A gate is called universal if any Boolean function (AND, OR, NOT) can be implemented using only that gate. Both NAND and NOR are universal.

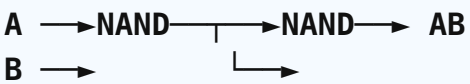
Q1. Prove the universality of NAND gate.

We need to construct NOT, AND, and OR using only NAND gates. (i) NOT using NAND: Connect both inputs of a NAND gate together.

```
A NAND A = (A·A)' = A'
```

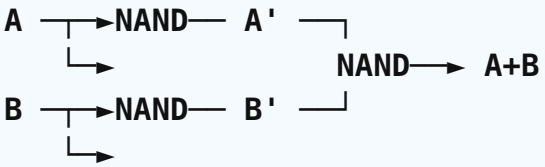
(ii) AND using NAND:
NAND followed by NOT (which is also NAND).

```
[(A·B)']' = AB
```



(iii) OR using NAND (using De Morgan):
A + B = (A'·B')'
First invert A and B (using NAND as NOT), then NAND them.

```
A + B = (A')' + (B')' = (A' · B')' = NAND(A', B')
```



Since NOT, AND, and OR can all be built from NAND gates alone, NAND is a universal gate. ∎

Q2. Prove the universality of NOR gate.

(i) NOT using NOR:

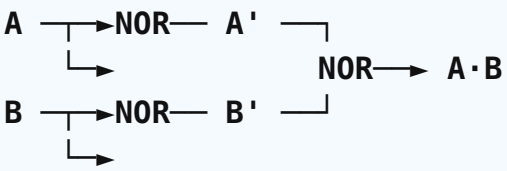
```
A NOR A = (A+A)' = A'
```

(ii) OR using NOR:

```
[(A+B)']' = A+B
```

NOR followed by NOT (NOR with same input). (iii) AND using NOR:
A · B = (A' + B')' (De Morgan)
First invert A and B using NOR-as-NOT, then NOR them.

```
A · B = NOR(A', B')
```



Hence, NOR is also a universal gate. ∎

Bonus: Word Problems (Real-World Logic Design)

Q1. Design a digital controller — Lab Entry System

A student is allowed to enter the university lab (E=1) if:
• Has Valid ID (I) AND Completed Safety Training (T) AND Not Late (L'), OR
• Has Valid ID (I) AND has Special Permission Slip (S). Boolean expression:

```
E = I·T·L' + I·S
```

Truth Table for E(I, T, L, S):

I	T	L	S	E
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

E = Σm(9, 11, 12, 13, 15) K-Map (rows IT; cols LS):

IT \ LS	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	1	1	1	0

10	0	1	1	0

Groupings:

• Group of 4: {m9, m11, m13, m15} → I·S

• Group of 2: {m12, m13} → I·T·L'

```
E = IS + ITL'
```

Logic Diagram:

2 AND gates + 1 OR gate. Need a NOT for L'.

Q2. Elevator door controller (3-story building)

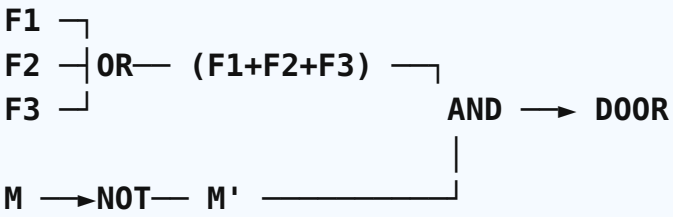
M = motion signal (1=moving, 0=stopped). F1, F2, F3 = floor signals (HIGH only at that floor). DOOR = 1 (open) when elevator is stopped (M=0) and at any floor. DOOR = M'·(F1 + F2 + F3)

Truth Table (only valid combinations: at most one F_i is HIGH):

M	F1	F2	F3	DOOR
0	0	0	0	0
0	1	0	0	1
0	0	1	0	1
0	0	0	1	1
1	×	×	×	0

```
DOOR = M'·(F1 + F2 + F3) = M'F1 + M'F2 + M'F3
```

Logic Diagram: Need 1 NOT (M'), 1 OR gate (3-input), 1 AND gate.



📝 Exam Tips & Quick Formula Sheet

Top 10 Must-Remember Formulas

1. **Half Adder: $S = A \oplus B$; $C = AB$**
2. **Full Adder: $S = A \oplus B \oplus C_{in}$; $C_{out} = AB + BC_{in} + AC_{in}$**
3. **Half Subtractor: $D = A \oplus B$; $B_{out} = A'B$**
4. **Full Subtractor: $D = A \oplus B \oplus B_{in}$; $B_{out} = A'B + A'B_{in} + BB_{in}$**
5. **De Morgan: $(A+B)' = A'B'$; $(AB)' = A'+B'$**
6. **Consensus: $A + A'B = A + B$**
7. **2's Complement: 1's complement + 1**
8. **4×1 MUX: $Y = S_1'S_0'I_0 + S_1'S_0I_1 + S_1S_0'I_2 + S_1S_0I_3$**
9. **BCD invalid codes: 1010 to 1111 (10 to 15)**
10. **Excess-3: Decimal digit + 3 → 4-bit binary**

Common Question Patterns Seen In Your Exams:

1. **Number conversions — always show every step.**
2. **K-map simplification — sometimes with don't-cares.**
3. **MUX design (8×1 from 2×1, or 16×1 from 4×1).**
4. **Boolean simplification — always quote the law (Distributive, Absorption, etc.).**
5. **Word problems (lab entry, elevator) — write expression first, then truth table → K-map → diagram.**
6. **Universality of NAND/NOR — always show NOT, AND, OR construction.**
7. **2-bit comparator / Priority encoder — full truth table is essential.**
8. **Subtraction by complement — show carry and discard rules clearly.**

Strategy on Exam Day:

- **Always show step-by-step working — don't skip steps.**
- **For K-maps, draw the map first, then circle groups, then write each product term.**
- **Label all gates clearly in logic diagrams.**
- **Verify your answer with truth table or arithmetic.**
- **Number conversions: write the division/multiplication table neatly.**
- **Time management: 4 questions × 30 min = 2 hours. Don't get stuck!**

Digital Logic Design — Complete Exam Preparation Guide
Covers all 14 topics from your syllabus, with patterns matching Metropolitan University CSE 211 exam.
Best of luck! 🎓